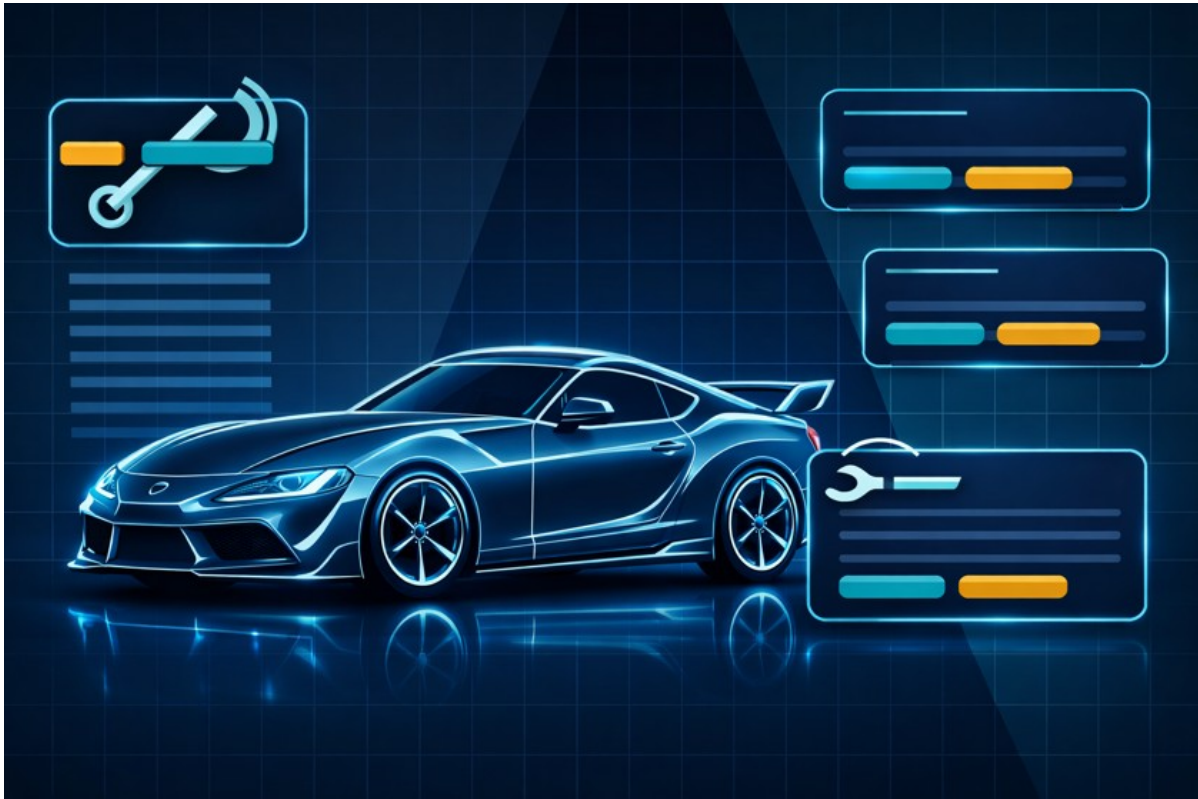


WerkstattPilot

Aufgabe 2: Projektumgebung definieren und dokumentieren



Projektbild / Profilbild: digitale Auftragsabwicklung für KFZ-Werkstätten

Angabe	Wert
Schule / Modul	HF Juventus Zürich - Software Engineering 1
Dozent	Arif Chughtai
Projekt	WerkstattPilot
Arbeitsgruppe	WerkstattPilot-Team
Autoren	Livio Luna, Dominic Lolos
Version	1.0
Datum	05.06.2026
Status	Abgabeversion

Inhaltsverzeichnis

Kapitel	Titel
1	Ziel und Gültigkeit der Projektumgebung
2	Richtlinien für Schreibweisen und Schreibstil
3	Ordnerstruktur
4	Organisation der Arbeitsgruppe
5	Eingesetzte Entwicklungssoftware
6	Installation und Konfiguration
7	Projektentscheidungen
8	Glossar
9	Rückmeldungen des Dozenten
10	Arbeitsprotokoll
11	Quellen und Referenzen

Dokumentzweck

Dieses Dokument beschreibt die Projektumgebung für das Softwareprojekt WerkstattPilot. Es definiert die gemeinsame Arbeitsweise, die eingesetzten Werkzeuge, die Ordnerstruktur, die wichtigsten Projektentscheidungen und die zentralen Fachbegriffe. Das Dokument wird während der Projektarbeit fortlaufend gepflegt.

Die Projektumgebung dient dazu, Dokumente, Modelle und Quelltexte einheitlich zu erstellen und nachvollziehbar abzulegen. Dadurch sollen Missverständnisse, doppelte Ablagen und uneinheitliche Bezeichnungen vermieden werden.

1. Ziel und Gültigkeit der Projektumgebung

Die Projektumgebung gilt für alle Arbeitsergebnisse im Projekt WerkstattPilot. Dazu gehören Projektdokumentation, UML-Modelle, Java-Quelltexte, Testdokumentation, Issue Tracking, Installationshinweise und Abgabeunterlagen.

Für die erste Iteration werden die beiden wichtigsten fachlichen Anforderungen umgesetzt: Auftrag anlegen sowie Auftragsübersicht mit Statusänderung. Alle weiteren Anforderungen, zum Beispiel Lagerverwaltung, Arbeitspositionen oder PDF-Belege, sind für spätere Iterationen vorgesehen.

Annahmen: Folgende Annahmen werden für diese Dokumentation verwendet und bei Bedarf später angepasst: Java JDK 21 LTS als verwendete Java-Version, IntelliJ IDEA als Entwicklungsumgebung, Git für Versionierung, Excel für Issue Tracking und keine echte Datenbank in der ersten Iteration.

2. Richtlinien für Schreibweisen und Schreibstil

2.1 Allgemeiner Schreibstil

- Die Projektdokumentation wird in Deutsch verfasst.
- Der Schreibstil ist kurz, formal und nachvollziehbar. Es werden keine langen Theorieabschnitte wiederholt.
- Fachbegriffe werden einheitlich verwendet. Für denselben Begriff wird nicht zwischen mehreren Schreibweisen gewechselt.
- Entscheidungen, Annahmen und offene Punkte werden sichtbar dokumentiert.
- Rechtschreibung, Gross- und Kleinschreibung sowie Java- und UML-Bezeichnungen werden vor der Abgabe geprüft.

2.2 Namenskonventionen für Dokumente und Ordner

Element	Regel	Beispiel
Ordner	nummeriert, klein, mit Unterstrich	02_dokumentation, 03_entwicklung
Dokumente	klein, Projektname, Aufgabe, Inhalt, Version	werkstattpilot_aufgabe2_projektumgebung_v1.0.docx
Diagramme	Diagrammtyp und Inhalt im Dateinamen	klassendiagramm_auftrag_v1.0.png
Versionen	Version mit vX.Y kennzeichnen	v1.0, v1.1
Abgaben	finale Dateien im Abgabeordner ablegen	werkstattpilot_aufgabe2_final.pdf

2.3 Namenskonventionen für Java und UML

Element	Regel	Beispiel
Packages	lowercase, keine Leerzeichen	ch.werkstattteam.werkstattpilot.business
Klassen	PascalCase, Singular	Auftrag, Kunde, Fahrzeug
Interfaces	PascalCase, fachlich klar	AuftragRepository
Methoden	camelCase, Verb plus Objekt	auftragAnlegen(), statusAendern()
Attribute	camelCase, fachlich eindeutig	auftragsNummer, kundenName
Konstanten	GROSSBUCHSTABEN_MIT_UNTERSTRICH	MAX_AKTIVE_AUFTRAEGE
UML-Klassen	Substantive in Kundensprache	Auftrag, Fahrzeug
Use Cases	Verb plus Objekt	Auftrag anlegen

2.4 Fachliche Sprache

Die fachlichen Klassen und Begriffe werden auf Deutsch benannt, weil das Projekt und die Aufgabenstellung in Deutsch geführt werden. Technische Standardbegriffe wie Repository, Factory, Mock-DB oder Unit-Test werden beibehalten, wenn sie im Unterricht oder in Java üblich sind.

3. Ordnerstruktur

Die Ordnerstruktur trennt Installation, Dokumentation, Entwicklung, Issue Tracking, Versionen, Abgaben und Backups. Dadurch bleibt nachvollziehbar, wo Arbeitsergebnisse abgelegt werden und welche Dateien abgaberelevant sind.

```
WerkstattPilot/  
  01_installation/  
    java_jdk/  
    intellij_idea/  
    visual_paradigm/  
    git/  
  02_dokumentation/  
    aufgabe_1_projektauftrag/  
    aufgabe_2_projektumgebung/  
    aufgabe_3_analyse/  
    aufgabe_4_design/  
    aufgabe_5_implementation/  
    diagramme/  
    protokolle/  
  03_entwicklung/  
    werkstattpilot/  
      src/main/java/ch/werkstattteam/werkstattpilot/  
        presentation/  
        business/  
        persistence/  
      src/test/java/ch/werkstattteam/werkstattpilot/test/  
        business/  
        persistence/  
  04_issue_tracking/  
  05_versionen/  
  06_abgabe/  
  07_backup/
```

Ordner	Zweck
01_installation	Installationsdateien, Setup-Hinweise und Konfigurationsnotizen.
02_dokumentation	Projektauftrag, Projektumgebung, Analyse, Design, Implementation, Diagramme und Protokolle.
03_entwicklung	IntelliJ-Projekt mit Java-Quelltexten, Packages und Tests.
04_issue_tracking	Excel-Datei für Aufgaben, offene Punkte, Fehler und Status.
05_versionen	Zwischenstände und wichtige freigegebene Versionen.
06_abgabe	Finale DOCX-, PDF- und Präsentationsdateien für die Abgabe.
07_backup	Sicherungen wichtiger Projektstände.

4. Organisation der Arbeitsgruppe

Die Arbeitsgruppe arbeitet als kleines Zweierteam. Die Aufgaben werden klar verteilt, Ergebnisse werden gegenseitig geprüft und wichtige Entscheidungen werden im Abschnitt Projektentscheidungen dokumentiert.

Person	Hauptrolle	Verantwortung
Livio Luna	Dokumentation, Analyse, Strukturierung, Qualitätssicherung	Erstellt und prüft Projektdokumente, formuliert fachliche Inhalte und kontrolliert Abgabekriterien.
Dominic Lolos	Technische Umsetzung, Entwicklung, Unterstützung Modellierung	Unterstützt bei Java-Umsetzung, technischer Struktur und Modellierung.
Beide	Abstimmung, Review, Präsentation, finale Abgabe	Stimmen Inhalte ab, prüfen Konsistenz und präsentieren die Projektarbeit gemeinsam.

4.1 Zusammenarbeit und Kommunikation

Bereich	Vorgehen
Kommunikation	Teams, persönliche Abstimmung, kurze Reviews
Austausch von Arbeitsergebnissen	Git für Code, gemeinsamer Projektordner für Dokumente und Abgaben

Planung	Arbeitsprotokoll und Issue Tracking in Excel
Qualitätssicherung	Gegenseitiger Review vor Abgabe, Prüfung der Bewertungskriterien
Dokumentation von Entscheiden	Entscheidungen werden mit Begründung und Auswirkung dokumentiert

4.2 Grober Projektplan

Phase	Inhalt	Status
Aufgabe 1	Projektauftrag und Anforderungen	abgeschlossen
Aufgabe 2	Projektumgebung definieren und dokumentieren	in Abgabeversion
Aufgabe 3	Objektorientierte Analyse: Fachklassenmodell und Zustandsmodell	nachführen
Aufgabe 4	Objektorientiertes Design: 3-Schichtenmodell, Patterns, Sequenzdiagramm	nachführen
Aufgabe 5	Java-Prototyp, Mock-DB, Unit-Tests, Javadoc und Endbenutzertest	nachführen

5. Eingesetzte Entwicklungssoftware

Die eingesetzte Software wird möglichst schlank gehalten. Für die erste Iteration werden nur Werkzeuge verwendet, die für Dokumentation, Modellierung, Java-Entwicklung, Versionierung und einfache Projektsteuerung notwendig sind.

Software	Version / Stand	Zweck	Installationsort	Website / Doku
Java JDK	21 LTS (Annahme)	Programmiersprache und Laufzeitumgebung für den Prototyp	lokaler Rechner	oracle.com/java oder adoptium.net
IntelliJ IDEA	lokal installierte Version	Entwicklungsumgebung für Java-Projekt, Packages und Tests	lokaler Rechner	jetbrains.com/idea
Git	2.x / lokale Version	Versionierung von Quelltext und wichtigen Projektständen	lokaler Rechner	git-scm.com
Visual Paradigm	lokal installierte Version	Erstellung von UML-Diagrammen	lokaler Rechner	visual-paradigm.com
Microsoft Word	Microsoft 365 / lokal	Projektdokumentation und Abgabe als DOCX/PDF	lokaler Rechner	support.microsoft.com
Microsoft Excel	Microsoft 365 / lokal	Issue Tracking, Arbeitsprotokoll und offene Punkte	lokaler Rechner	support.microsoft.com
JUnit	5.x (spätere Aufgabe)	Unit-Tests für Business- und Persistence-Schicht	IntelliJ-Projekt	junit.org

Hinweis: Die exakten lokalen Versionsnummern von IntelliJ IDEA, Visual Paradigm und Git werden beim Setup bzw. vor der finalen Abgabe nochmals geprüft und im Arbeitsprotokoll nachgeführt.

6. Installation und Konfiguration

Die folgende Checkliste beschreibt die notwendigen Schritte für Installation, Konfiguration und Nutzung der Entwicklungssoftware. Ziel ist, dass beide Teammitglieder mit derselben Projektstruktur arbeiten können.

6.1 Java JDK

Schritt	Richtlinie / Kontrolle	Status
JDK installieren	Java JDK 21 LTS installieren.	erledigt / prüfen
JAVA_HOME prüfen	JAVA_HOME auf das installierte JDK setzen, falls notwendig.	prüfen
Version prüfen	Terminal: java -version ausführen.	prüfen
IntelliJ verbinden	Projekt-SDK in IntelliJ auf das installierte JDK setzen.	prüfen

6.2 IntelliJ IDEA

Schritt	Richtlinie / Kontrolle	Status
IDE installieren	IntelliJ IDEA installieren und starten.	erledigt / prüfen

Neues Projekt erstellen	Java-Projekt WerkstattPilot mit JDK 21 erstellen.	prüfen
Packages anlegen	presentation, business und persistence gemäss 3-Schichtenstruktur anlegen.	prüfen
Tests vorbereiten	test.business und test.persistence für JUnit-Tests vorbereiten.	spätere Aufgabe
Run-Konfiguration	Applikationsklasse bzw. ConsoleClient für späteren Prototyp vorbereiten.	spätere Aufgabe

6.3 Git

Schritt	Richtlinie / Kontrolle	Status
Git installieren	Git lokal installieren.	erledigt / prüfen
Repository initialisieren	Im Entwicklungsordner ein Git-Repository anlegen.	prüfen
Commit-Regeln nutzen	Kurze, verständliche Commit-Messages schreiben.	laufend
Nicht versionieren	Build-Ordner, IDE-Cache und temporäre Dateien nicht versionieren.	laufend
Backup sicherstellen	Wichtige Zwischenstände zusätzlich im Backup-Ordner sichern.	laufend

6.4 Visual Paradigm

Schritt	Richtlinie / Kontrolle	Status
Tool installieren	Visual Paradigm installieren oder vorhandene Installation verwenden.	erledigt / prüfen
Projekt anlegen	UML-Projekt WerkstattPilot anlegen.	prüfen
Diagramme strukturieren	Diagramme nach Aufgabe und Diagrammtyp benennen.	laufend
Export	Diagramme als PNG oder PDF in den Dokumentationsordner exportieren.	laufend

6.5 Word und Excel

Schritt	Richtlinie / Kontrolle	Status
Word-Vorlage nutzen	Einheitliche Überschriften, Tabellen und Dateinamen verwenden.	laufend
PDF erstellen	Finale Abgabe zusätzlich als PDF exportieren.	laufend
Excel Issue Tracking	Offene Punkte, Aufgaben, Fehler und Status in Excel pflegen.	laufend
Arbeitsprotokoll	Datum, Beteiligte, Tätigkeit, Ergebnis und nächster Schritt festhalten.	laufend

7. Projektentscheidungen

Projektentscheidungen werden dokumentiert, damit spätere Änderungen nachvollziehbar bleiben. Die Tabelle enthält Entscheidung, Begründung und Auswirkung auf die Projektarbeit.

Datum	Entscheid	Begründung	Auswirkung
05.06.2026	Projektname WerkstattPilot	Der Name beschreibt den fachlichen Zweck der Anwendung klar.	Einheitlicher Name in Dokumenten, Diagrammen und Code.
05.06.2026	Screenshot als Projektbild verwenden	Das Bild zeigt die digitale Werkstatt- und Auftragsumgebung visuell passend.	Verwendung als Profilbild/Titelbild in der Dokumentation.
05.06.2026	Iteration 1 umfasst UC1 und UC2	Auftrag anlegen und Statusübersicht haben hohen fachlichen Nutzen.	Alle Analyse-, Design- und Implementationsarbeiten fokussieren auf diese Funktionen.
05.06.2026	Deutsch als Fachsprache	Aufgabenstellung und Fachdomäne sind in Deutsch.	Klassen und Fachbegriffe werden auf Deutsch benannt.
05.06.2026	Java und IntelliJ IDEA verwenden	Java passt zum Unterricht und IntelliJ ist die gewählte Entwicklungsumgebung.	Prototyp wird als Java-Projekt in IntelliJ umgesetzt.
05.06.2026	Keine echte Datenbank in Iteration 1	Der Prototyp soll zuerst einfach und testbar bleiben.	Datenhaltung erfolgt über Mock-DB bzw. Java

			Collections im Hauptspeicher.
05.06.2026	Git für Versionierung	Änderungen am Code sollen nachvollziehbar bleiben.	Regelmässige Commits und klare Commit-Messages.
05.06.2026	Excel für Issue Tracking	Für die Projektgrösse reicht ein einfaches, transparentes Tracking.	Aufgaben, Fehler und offene Punkte werden in einer Excel-Liste gepflegt.
05.06.2026	Visual Paradigm für UML	Das Tool unterstützt die im Unterricht geforderten UML-Diagramme.	Diagramme werden dort erstellt und in die Dokumentation exportiert.
05.06.2026	3-Schichtenstruktur vorbereiten	Die spätere Umsetzung verlangt Presentation, Business und Persistence.	Packages werden entsprechend strukturiert.

7.1 Umgang mit der Datenbank-Frage

In der ersten Projektidee war eine lokale Datenhaltung mit SQLite oder PostgreSQL möglich. Für die aktuelle Iteration wird dies bewusst nicht umgesetzt. Der Prototyp verwendet stattdessen eine Mock-DB im Hauptspeicher. Eine echte lokale Datenbank bleibt eine Option für spätere Iterationen, gehört aber nicht zum aktuellen Abgabebumfang.

8. Glossar

Begriff	Beschreibung	Synonyme
WerkstattPilot	Name des zu entwickelnden Softwaresystems für digitale Werkstattaufträge.	Anwendung, System
Auftrag	Fachliches Objekt, das einen Werkstattauftrag für ein Fahrzeug beschreibt.	Werkstattauftrag
Kunde	Person oder Organisation, die einen Auftrag für ein Fahrzeug erteilt.	Auftraggeber
Fahrzeug	Fahrzeug des Kunden, für das ein Auftrag erfasst wird.	Auto, Kundenfahrzeug
Beanstandung	Vom Kunden gemeldetes Problem oder gewünschte Arbeit am Fahrzeug.	Problem, Anliegen
Termin	Geplanter Zeitpunkt für Annahme oder Bearbeitung des Auftrags.	Datum, Werkstatttermin
Status	Bearbeitungszustand eines Auftrags, zum Beispiel offen, in Arbeit oder fertig.	Auftragsstatus
Auftragsübersicht	Ansicht aller relevanten Aufträge mit Status und wichtigen Informationen.	Dashboard, Übersicht
Iteration 1	Erste abgegrenzte Umsetzungsphase mit Auftrag anlegen und Auftragsübersicht.	erste Lieferung
Use Case	Beschreibt eine fachliche Funktion aus Sicht eines Benutzers.	Anwendungsfall
Mock-DB	Vereinfachte Datenhaltung ohne echte Datenbank, zum Beispiel im Arbeitsspeicher.	In-Memory-Datenhaltung
In-Memory	Daten werden während der Programmausführung im Hauptspeicher gehalten.	Hauptspeicher
Presentation-Schicht	Schicht für Benutzerschnittstelle oder Konsolenausgabe.	UI-Schicht
Business-Schicht	Schicht für Fachlogik und fachliche Regeln.	Logikschicht
Persistence-Schicht	Schicht für Speichern und Lesen von fachlichen Objekten.	Datenzugriffsschicht
Repository	Schnittstelle oder Klasse für den Zugriff auf gespeicherte fachliche Objekte.	Datenzugriff
Factory	Erzeugt Objekte oder konkrete Implementierungen zentral.	Erzeugerklasse
Issue Tracking	Liste zur Verwaltung von Aufgaben, Fehlern und offenen Punkten.	Aufgabenliste
Git	Werkzeug zur Versionierung von Quelltexten und Projektständen.	Versionsverwaltung
Javadoc	Dokumentationsstandard für Java-Quelltext und API-Dokumentation.	Java-Dokumentation

9. Rückmeldungen des Dozenten

Zum Zeitpunkt der Erstellung dieser Abgabeverion liegen keine Rückmeldungen des Dozenten zu Aufgabe 2 vor. Rückmeldungen werden in der folgenden Tabelle festgehalten und bei der Weiterbearbeitung nachgeführt.

Datum	Rückmeldung	Umsetzung	Status
05.06.2026	Keine Rückmeldung vorhanden	Keine Anpassung notwendig	offen für spätere Ergänzung

10. Arbeitsprotokoll

Das Arbeitsprotokoll hält die wichtigsten Arbeitsschritte fest. Es wird während der weiteren Projektarbeit ergänzt und dient als Nachweis für Arbeitsorganisation und Nachvollziehbarkeit.

Datum	Beteiligte	Tätigkeit	Ergebnis	Nächster Schritt
05.06.2026	Livio Luna	Aufgabe 2 neu strukturiert	Dokumentaufbau gemäss Bewertungskriterien erstellt	DOCX und PDF finalisieren
05.06.2026	Livio Luna	Projektentscheide festgelegt	IntelliJ, Java, Git, keine DB, Excel Issue Tracking und Deutsch als Fachsprache bestätigt	Entscheide im Dokument nachführen
05.06.2026	Livio Luna	Projektbild ausgewählt	Screenshot als Projekt-/Profilbild übernommen	Titelbereich aktualisieren
laufend	Livio Luna / Dominic Los	Review und Ergänzungen	Offene Punkte und Rückmeldungen werden ergänzt	Arbeitsprotokoll weiterführen

11. Quellen und Referenzen

Quelle	Verwendung im Projekt
Aufgabenblatt Aufgabe 1	Projektauftrag und Anforderungen für WerkstattPilot.
Aufgabenblatt Aufgabe 2	Vorgaben zur Projektumgebung, Bewertungskriterien und Dokumentationsumfang.
Aufgabenblatt Aufgabe 5	Vorgaben zur späteren Java-Implementation, insbesondere 3-Schichtenstruktur und Mock-DB.
Java-Dokumentation	Referenz für Java-Sprache, JDK und Javadoc.
IntelliJ IDEA Dokumentation	Referenz für Projektkonfiguration und Java-Entwicklung.
Git-Dokumentation	Referenz für Versionierung und grundlegende Git-Befehle.
Visual Paradigm Dokumentation	Referenz für Erstellung und Export von UML-Diagrammen.

Hinweis: Diese Referenzen werden nicht als Theorieabschnitte wiederholt, sondern nur als Grundlage für die konkrete Projektumgebung verwendet.